

Базы данных

Лекция 01 – Пользовательский интерфейс

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2025

2025.09.02

Оглавление

Пример базы данных.....	3
Структура клиентского приложения.....	4
Общее.....	4
Замечания относительно схемы базы данных.....	7
Принципы CUA.....	8
Простота использования.....	8
Единообразие интерфейса (Interface Consistency).....	9
Аспекты пользовательского интерфейса.....	10
Передача знаний (Transfer of Learning).....	11
Модальное поведение (Modal Behavior).....	12
Исследование и откат (Exploring and Backing Out).....	13
Припоминание против распознавания (Recall Versus Recognition).....	14
Пути новичка и эксперта (Novice and Expert Paths).....	15
Контекстная информация (Contextual Information).....	16
Основы интерфейса приложений.....	17
Компоненты панели CUA.....	18
Особенности формы TABLE (пример).....	20
Особенности формы FORM.....	24
Общие.....	24
Навигация по форме.....	26
Обработка полей ввода.....	27
Создание новой записи.....	29

Пример базы данных

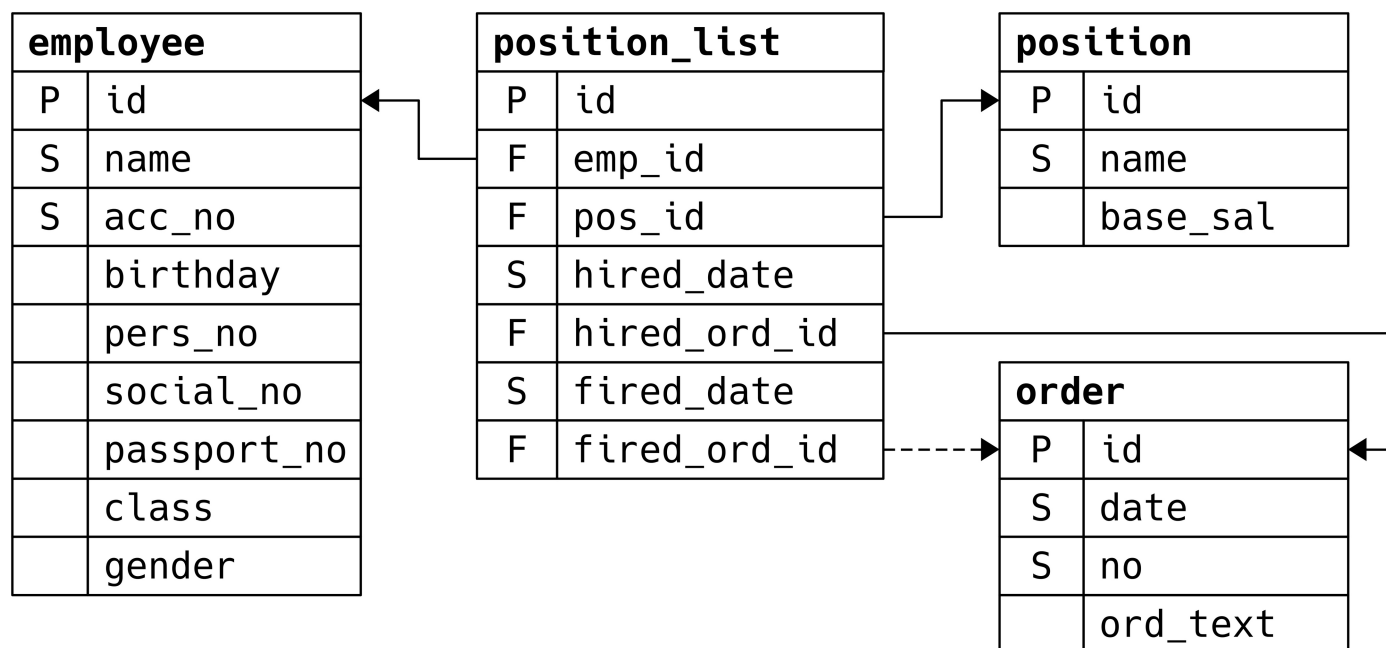


Рисунок 1 — Фрагмент базы данных

emp:employee – Сотрудник

emp.name – Фамилия, имя и отчество
emp.no – Учетный номер
emp.pers_no – Персональный номер
emp.social_no – Номер соцстраха
emp.passport_no – Номер паспорта
emp.class – Класс служащего

pos:position – Должность

pos.name – Наименование должности
pos.base_sal – Базовая ставка

ord:order – Приказ

ord.date – Дата приказа
ord.order – Текст приказа

plist:position_list – Послужной список

plist.emp_id – Ссылка на сотрудника
plist.pos_id – Ссылка на должность
plist.hired_date – Дата назначения
plist.hired_ord_id – Ссылка на приказ
plist.fired_date – Дата увольнения
plist.fired_ord_id – Ссылка на приказ

Структура клиентского приложения

Общее

Все существующие базы данных, за исключением использующихся для хранения телеметрии, наполняются и обновляются в интерактивном режиме (оператором). Соответственно, приложение, предоставляющее оператору интерфейс к базе данных, должно быть удобным и эффективным прежде всего для оператора, и обеспечивать хорошую его производительность при минимальной утомляемости. Требования к интерфейсу должны исходить прежде всего из физиологии и психологии здорового человека и, по возможности, ориентироваться на продвинутого пользователя/оператора.

Это требование не отрицает наличия у приложения возможностей, облегчающих работу начинающим пользователям (всплывающие подсказки, справка, детализированные контекстные меню, дополнительные диалоги на подтверждение действий). Иногда нужно подробное изучение документации, а в некоторых случаях могут потребоваться специальные приложения-тренажеры, работающие на тестовых данных. Тем не менее, все, что мешает работе продвинутого пользователя, не следует использовать. Это касается в первую очередь анимации, являющейся одной из причин психологической усталости операторов. В этой же связке следует рассматривать и стабильность дизайна интерфейса. Нет никакой необходимости постоянно менять интерфейс, если не меняется схема и бизнес-логика.

Если интерактивные операции обновления баз данных выполняются на ПК (ноутбуках), обладающих достаточно большим монитором, клавиатурой и указательным устройством с разрешением не хуже экранного пиксела (мышка, планшет, тачпад), интерфейс таких приложений должен четко соответствовать возможностям ввода/вывода ПК.

Если эти операции выполняются с мобильных устройств, не обладающих физической клавиатурой и имеют указательное устройство в виде пальца, интерфейс для них должен разрабатываться с учетом именно этих возможностей. В любом случае интерфейс приложения должен всегда соответствовать возможностям ввода/вывода устройства без каких-либо компромиссов.

1) Дизайн приложения для компьютера, обладающего развитыми средствами ввода/вывода (ПК, ноутбук), должен следовать рекомендациям CUA — Common User Access. Это документ, разработанный IBM с привлечением ортопедов-травматологов, неврологов, окулистов, хирургов, психологов и даже страховщиков, предназначен для разработки эффективных и производительных интерфейсов, обеспечивающих минимальную утомляемость оператора в процессе работы в течение полного рабочего дня, а также отсутствие необходимости выплачивать регрессы ослепшим или заработавшим CTS¹ в результате работы инклюзивных дизайнеров.

2) Каждая таблица, содержащая информацию, представляющую интерес для пользователя/оператора, должна, как минимум, иметь следующие диалоговые окна в типажах CUA:

форма TABLE — окно для отображения и навигации по таблице (далее может использоваться слово «таблица», смысл которого устанавливается из контекста);

форма FORM — форма для детального просмотра полей записи таблицы и ее редактирования (далее FORM или «форма»).

3) Приложение должно быть MDI и иметь элемент Pull-Down меню, содержащее вызов формы TABLE для каждой из таблиц, являющихся справочниками (LUT — LookUp Table), поскольку часто они обновляются отдельно.

4) Дополнительно могут содержаться:

- комбинированные панели, содержащие компоненты формы и компоненты таблицы;
- Pop-Up меню, содержащие действия в контексте вызова.

5) Любое приложение всегда имеет хотя бы одну основную таблицу.

Если основная таблица одна, ее форма TABLE автоматически должна вызываться при старте приложения. Если основных таблиц несколько, Pull-Down меню приложения должно иметь элемент с вызовами для них форм TABLE. При старте такого приложения либо вызывается последняя активная таблица из основных, либо таблица по умолчанию, либо ни одна не вызывается. Выбор варианта может настраиваться.

1) Carpal Tunnel Syndrome – синдром запястного канала

6) Таблицы выводятся в отсортированном порядке одного из ключей сортировки (вторичные ключи, обозначаемые на реляционных схемах как 'S').

Если таблица имеет несколько ключей сортировки, должна быть возможность один из ключей назначить ключом по умолчанию. Если ключей сортировки несколько, таблица при отображении сортируется либо по последнему использовавшемуся ключу сортировки, либо по ключу, который назначен ключом по умолчанию. Выбор варианта должен настраиваться.

Поле S-ключа должно выводиться в самом левом столбце формы «таблица».

7) В ряде случаев необходимо отображать в форме TABLE различный набор столбцов, поскольку все значимые столбцы могут не помещаться по горизонтали. В таком случае могут использоваться следующие варианты:

- дискретное «скроллинг» с заменой полей, однако при этом ведущий S-столбец должен оставаться всегда на месте (слева);

- использовать виджет «Вкладки», переключающий наборы полей.

Следует избегать скроллинга в TABLE по горизонтали с использованием скролл-бара.

8) Использовать «перелом» строки и выводить ее в TABLE в несколько строк запрещено.

9) Панель FORM следует проектировать таким образом, чтобы исключить скроллинг информационной области по вертикали. Если количество информации, которую следует отобразить, превышает емкость экрана, следует использовать группировку полей и вкладки в соответствии с группировкой.

10) Следует стремиться к тому, чтобы приложение отвечало на действия пользователя не позже, чем через 100 мс². Если это невозможно (операции ввода-вывода, обработка запроса, ...), следует выводить панель ожидания (часы, термометр, и пр.) — оператор должен быть проинформирован о том, что система выполняет его запрос и ему придется подождать. Использование анимации и других способов сокрытия задержек ответа, запрещено.

2) 100 мс – предел интервала, который человеком ощущается, как «сразу».

11) Все панели и прочие формы UI должны отображаться сразу же готовыми к использованию. Задержки между появлением на экране и готовностью к вводу не должны превышать 100 мс. Это легко достигается использованием статических компилируемых форм.

Не все UI-фреймворки обеспечивают реализацию требований 10 и 11, соответственно, они не могут использоваться при разработке приложений, с которыми работает оператор полный рабочий день.

Замечания относительно схемы базы данных

Первичные ключи записей не должны нести никакой информации, которая могла бы представлять интерес для пользователя. Из этого следует:

- первичные ключи не должны отображаться в приложении – они используются только для идентификации записей в таблицах;
- несмотря на уникальность каких-либо информационных полей, они не должны использоваться в качестве первичных ключей записи — они могут впоследствии измениться, что потребует остановки базы данных для возврата согласованности;
- в качестве первичного ключа следует использовать уникальное в рамках таблицы автоинкрементируемое число, т.н. синтетический ключ (автоинкрементный тип SERIAL).

Вторичные ключи, как не несущие никакой информации, которая могла бы представлять интерес для пользователя, также не должны отображаться в приложении, поскольку они служат исключительно для связи таблиц и содержат значения первичных ключей, которые никому не интересны.

Принципы CUA

Простота использования

- 1) следует делать интерфейсы единообразными — если интерфейсы единообразны, то, освоив один интерфейс, пользователь без особых усилий сможет разобраться и в смежных;
 - 2) следует выбирать объекты, интуитивно понятные пользователю, для этого необходимо, чтобы дизайн был основан на концептуальных сущностях, знакомых пользователю;
 - 3) следует выбирать интуитивно понятные операции над объектами, например, «создать (create)», «вставить (insert)», «добавить (append)», «изменить (change)», «отредактировать (edit)», «удалить (delete)», «выбрать (select)» и «работать с (work with)», по возможности следует использовать простые стандартные выражения;
 - 4) следует руководствоваться принципом «объект-действие» — как правило, пользователь должен выбрать объект, а затем действие для него (объектный подход для реализации интерфейса);
 - 5) следует предусмотреть возможность отката действий до их финального подтверждения — пользователь должен иметь возможность исследовать систему без серьёзных последствий;
 - 6) следует обеспечить обратную связь для уверенности пользователя на каждом этапе (информационная область панели);
 - 7) следует предоставить онлайн-справку и поисковые индексы.
- Чтобы сделать систему эффективной в использовании, необходимо учесть следующее:
- следует использовать максимально простую структуру;
 - следует избегать необходимости запоминания информации пользователем;
 - следует избегать режимов работы приложения;
 - следует предоставлять базовые и экспертные пути решения задач (меню и клавиатура);
 - следует сделать наиболее распространенный путь путем по умолчанию (автофокус в формах);
 - следует предоставлять систему готовой к «боевому» использованию;
 - следует предоставлять дополнительные возможности только по запросу.

Единообразие интерфейса (Interface Consistency)

Нижеизложенное не касается интерфейсов пользователя с сенсорным экраном (touchscreen) и тачпадов, поддерживающих жесты (gesture touchpad).

В соответствии с требованиями SAA компании IBM единообразие подразделяется на три уровня:

Физический — фактическое расположение физических элементов, таких как клавиатура, монитор, устройство указания (мышь, тачпад) должно быть единообразным. В принципе, сегодня на рабочих местах операторов физическая единообразность, в основном, обеспечивается аппаратным обеспечением.

Синтаксический — использование элементов интерфейса должно быть единообразным. Например, язык представления³ (например, расположение и заполнение панелей), язык действий (например, Enter = Принять изменения) и CL-синтаксис⁴ должны быть единообразными.

Семантический — значение элементов интерфейса и их взаимодействие должны быть единообразными. Например, команда «Выход (Esc/Exit)» всегда должна возвращать оператора с панели без каких либо обновлений.

3) presentation language

4) синтаксис командного языка

Аспекты пользовательского интерфейса

Элементы CUA в IBM SAA включают стандарты для следующих аспектов поведения интерфейса.

Следует стремиться к их согласованности:

- управление панелью (panel management);
- управление активностью (activity management);
- раскладка и использование клавиатуры (keyboard layout and usage);
- взаимодействие с панелью (дисплеем) (panel (display) interaction);
- действие выбора (selection action);
- сообщения и подсказки (messages and prompts);
- цвет и выделение (color and emphasis);
- справка (help);
- поддержка национальных языков (national language support);
- терминология для конечных пользователей (terminology for end users).

Передача знаний (Transfer of Learning)

У человека есть врождённая способность обобщать на основе смежных случаев — она используется для изучения языка. Следует применять этот навык при использовании компьютерных интерфейсов. Например, если человек знает, что существует команда «Удалить запись» для удаления записей, и желает удалить новый тип объекта, скажем, «роль», он ожидает, что команда «Удалить роль» удалит ее.

Поэтому следует выбирать операции с интуитивно понятными словами, выражениями и метафорами. Большинство операций можно представить в виде относительно небольшого числа примитивных операций, например, создание, изменение, удаление, перемещение, слияние и разделение, которые интуитивно понятны пользователю.

Еще один аспект касается визуальной идентификации объектов и действий с использованием иконок и прочих символических значков. Дело в том, что текстовая идентификация перманентна (язык меняется медленно), а идентификация с помощью символических изображений, крайне скоротечна и не должна использоваться без сопровождающего текста.

Модальное поведение (Modal Behavior)

Пользователям часто приходится переключаться между задачами. Желательно не ограничивать пользователя в его дальнейших действиях. В частности, следует избегать «режимов» и «блок-схем» действий.

Программа демонстрирует «модальное поведение», если ограничивает пользователя ограниченным набором действий, которые определяются предыдущими действиями (блок-схема).

Режим требует от пользователя выполнения определённых действий в определённой последовательности для выхода из него.

Хотя практически невозможно избежать режимов из-за иерархической модели вызовов, всё же следует стараться минимизировать их влияние, в частности, должен быть обеспечен полный откат всей последовательности модальных действий.

Исследование и откат (Exploring and Backing Out)

Самый простой способ понять, как работает система, — это попробовать разные варианты и посмотреть, что из этого получится. Исследование — гораздо более естественный механизм обучения, чем абстрактное постижение (например, предварительное прочтение инструкций⁵).

Чтобы пользователь мог безопасно исследовать приложение:

- следует предоставить возможность «отката», это позволит пользователю выйти из функций, не затрагивая данные;
- следует подчеркивать точки невозврата, например, запросом на подтверждение или меню выхода.

Вся обработка обновлений должна выполняться сразу после подтверждения. Не должно быть промежуточных отображений, из которых единственным выходом будет дальнейшее обновление базы данных, поскольку это представляет собой модальное поведение.

При проектировании с возможностью отката могут быть полезны средства управления фиксацией изменений, поскольку несколько обновлений можно сгруппировать для выполнения по принципу «все или ничего».

5) В некоторых случаях работа без внимательного изучения инструкций и руководств невозможна. Как говорится, первая фраза в инструкции «Ну что, слоупок, уже сломал?»

Припоминание против распознавания (Recall Versus Recognition)

Распознавать информацию легче, чем её вызывать. Например, даже если человек не всегда может вспомнить наименование книги по запросу, он всё равно может узнать его среди связанных имён.

Поэтому жесткие запросы на получения конкретной информации должны иметь шестой приоритет, а наиболее приоритетным должен быть функционал выбора из лексически связанных наименований. Также имеет смысл делать каталоги тегов для этой цели.

Везде, где есть выбор значений для ввода, следует предоставлять функции запроса для отображения списка доступных вариантов (локаторное поле и LUT).

Пути новичка и эксперта (Novice and Expert Paths)

Требования пользователя, регулярно использующего систему, существенно отличаются от требований пользователя, впервые работающего с ней, или пользователя, использующего её время от времени.

Эксперт (продвинутый пользователь) сохраняет гораздо больше знаний о том, как использовать систему, и ему, как правило, требуются максимально эффективные пути выполнения обычных задач.

Новичку же потребуется больше поддержки.

Поэтому необходимо проектировать системы так, чтобы они предусматривали как «быстрый», так и «медленный» путь.

Медленный путь, обычно включающий меню и функции запросов, должен позволять конечному пользователю использовать функции запросов, чтобы убедиться в правильности своих действий.

Система с быстрым путём должна обеспечивать максимально быстрый переход между транзакциями (использование клавиатурных комбинаций позволяет избежать постоянных переносов руки с клавиатуры на устройство указания, избегая тем самым возможности заработать CTS).

На всех уровнях меню следует предусмотреть возможность вызова пункта с помощью клавиатурной комбинации. Если действие не входит в перечень часто используемых (например, выделить, вырезать, вставить) и им не назначены общесистемные клавиатурные комбинации, CUA предлагает устанавливать «активные символы» в наименованиях пунктов меню и активировать их, префиксируя с помощью нажатия и удержания клавиши <Alt>. Таким образом продвинутый пользователь может использовать цепочку нажатий для выполнении задачи вместо использования указательного устройства. Именно поэтому все события, инициированные устройством указания, в том числе и «drag-n-drop» должны проксироваться через клавиатурные комбинации. Это гарантирует существование полного экспертного пути.

Контекстная информация (Contextual Information)

Сложно поддерживать концентрацию внимания в течение длительного времени. При использовании сложной системы пользователи могут теряться в понимании своего местонахождения, особенно если их что-то отвлекает. Необходимо предоставить информацию, напоминающую пользователям, где они находятся и что делают.

Эта информация должна быть стандартизирована и располагаться в одном и том же месте (например, в заголовках и областях инструкций на панели), чтобы её можно было игнорировать без необходимости. Наиболее полезной информацией для установления контекста, как правило, является информация о любых непосредственно связанных сущностях, например, о клиенте, для которого оформлен заказ, или об отделе, к которому принадлежит сотрудник. Наличие такой информации позволяет пользователю с первого взгляда определить, что он делает, особенно при возвращении после перерыва. Связи между панелями должны соответствовать структуре данных, чтобы это было возможно.

Основы интерфейса приложений

Существует несколько общих принципов проектирования пользовательских интерфейсов, с которыми следует ознакомиться. Пользовательский интерфейс (UI) — это общий термин, используемый для описания тех аспектов компьютерной системы, которые видны пользователю и с которыми он взаимодействует при ее использовании. Для описания UI необходимо учитывать как статические компоненты, такие как раскладки клавиатуры, дизайн панелей и отчетов, так и динамические компоненты, такие как реакция программ на действия пользователя.

В документе IBM Common User Access (CUA) изложены подробные правила внешнего вида и поведения пользовательских интерфейсов как для текстовых, так и для графических терминалов.

Интерфейс является «чистым» графическим, если возникает необходимость манипулировать графическими объектами, для чего обязательно требуется устройство указания (САПР, например).

Если без указательного устройства можно обойтись, интерфейс является текстовым независимо от того, работает ли устройство вывода (экран) в графическом режиме, либо в текстовом.

Именно невозможность обойтись без использования устройства указания разделяет интерфейсы на графические и неграфические (текстовые). Этот факт позволяет объединить в рамках CUA требования к UI как для текстовых, так и для графических терминалов.

Для текстового интерфейса справедливо правило, состоящее в том, что все действия, которые можно выполнить в приложении, должны инициироваться командами с клавиатуры и это должен быть основной интерфейс. Использование указательного устройства является вторичным и его действие всего-лишь инициирует соответствующее клавиатурное событие. Данный подход позволяет иметь всего один вариант вызова процедуры, обрабатывающей событие.

Этот подход накладывает существенные ограничения на использование фреймворков, предназначенных для программирования UI. Как правило, все базовые фреймворки, прямо опирающиеся на библиотеки взаимодействия с оконными менеджерами, инструментами рисования и устройствами ввода, удовлетворяют данному требованию. Чем меньше уровней абстракции, тем меньше функциональных ограничений и тем проще реализовать удобный и быстрый UI.

Компоненты панели CUA

Панели следует рассматривать как состоящие не из низкоуровневых элементов, с помощью которых они составляются (буквальные символы, поля, информационные элементы), а из логических компонентов более высокого уровня, таких как заголовок, строка пояснения к клавише команды, кнопки действий и прочие поля, каждое из которых сопровождается текстом. Это позволяет установить эквивалентность и, следовательно, согласованность между различными типами панелей и даже различными типами рабочих станций, например, между станциями с текстовым экраном и графическим. Обобщенный стандарт SAA CUA для компоновки панелей выглядит следующим образом:

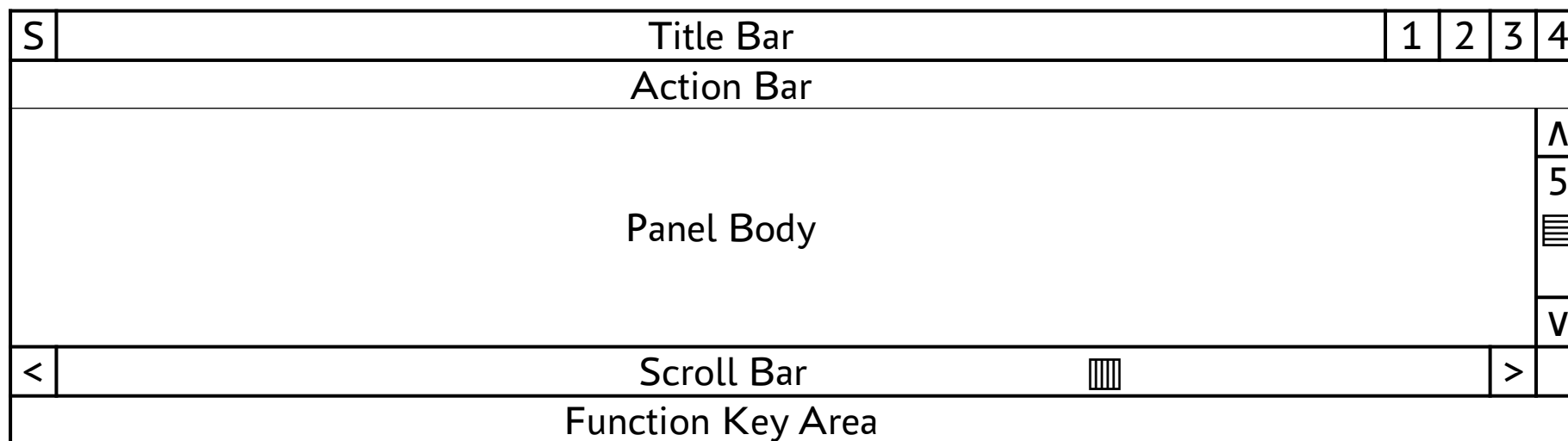


Рисунок 2 — Компоновка панели CUA.

Условные обозначения: 1, 2, 3, 4 — области для управления состоянием окна (заккрыть, скрыть, развернуть и пр., при необходимости это делать); S — область системного меню (при необходимости использовать соответствующие функции); Title Bar — заголовок окна/панели; Action Bar — область кнопок/меню для predetermined функций; Panel Body — тело панели; Scroll Bar — полоса горизонтальной прокрутки; 5 — полоса вертикальной прокрутки; Function Key Area — область функциональных действий.

Стандарт компоновки панелей CUA можно трактовать либо строго, исключая все вспомогательные данные, такие как дата, время или идентификация оператора, либо более гибко, сохраняя компоненты CUA на стандартных местах, но добавляя дополнительную информацию.

Все элементы панели CUA являются обязательными (за исключением полосок прокрутки) и статическими. Если содержимое тела панели полностью размещается в своей области по горизонтали и/или вертикали, соответствующие полоски прокрутки следует опустить.

Динамического формирования панелей следует избегать — если предполагается, что когда-то информация, отображаемая в панели, потребует каких-то особых элементов управления, они должны обязательно присутствовать, но быть неактивными или невидимыми.

Динамическое формирование панелей требует значительных вычислительных ресурсов и существенно препятствует соблюдению требования «задержка ответа не более 0,1 с».

Содержимое тела панели не должно меняться в зависимости от того, какие элементы управления видны — геометрия области тела панели, если размеры панели не меняются, должна сохраняться неизменной. Следствие: прежде всего это касается полосок прокрутки, которые либо должны всегда обязательно присутствовать, либо всегда отсутствовать. Присутствующие полоски в случае отсутствия необходимости в прокрутке (материал полностью уместается в области тела) должны быть неактивны, в том числе и визуально.

Особенности формы TABLE (пример)

Служащие (редактирование)

Фамилия Имя Отчество	Учетный №	Класс	
			▲
			▼

Локатор

Редактирование Вставка Удаление Печать

1) Форма должна вызываться для работы с таблицей базы данных в следующих режимах:

- просмотр с редактированием;
- просмотр без редактирования;
- удаление;
- выбор записи.

2) Заголовок окна должен отображать имя таблицы и режим ее работы.

3) Форма во всех режимах содержит виджет, отображающий диапазон записей таблицы БД в порядке установленной сортировки.

Если строки таблицы не умещаются целиком в область отображения, она должна быть скроллируемой по вертикали (шаг – строка).

Данный виджет должен иметь визуальный курсор — полосу выбора, синхронизированную в отношении доступа к указываемой записи и курсора в базе данных.

4) Во всех режимах **может** присутствовать локатор – поле, ввод в которое устанавливает курсор в таблице на соответствующую запись. Если перемещение курсора выбора происходит в пределах отображаемой области и устанавливается не на первую/последнюю из строк отображения, последнее не изменяется. В противном случае содержимое отображения изменяется таким образом, чтобы курсор выбора располагался со второй до предпоследней строки.

5) Общие действия в формах «Таблица» предполагают перемещение полосы выбора (курсора) в области отображения (буфер) и отображаемом диапазоне с помощью клавиатуры и указателя, для чего используются навигационные клавиши и их комбинации с модификатором «Ctrl»:

Up/Down — перемещение курсора вверх/вниз на одну строку;

PageUp/PageDown — листание диапазона вверх/вниз на одну страницу отображения;

Ctrl+Home/Ctrl+End — перемещение курсора в начало/конец отображаемого диапазона;

Home/End — перемещение курсора в начало/конец буфера.

Указатель на форме таблица может кликать на кнопках, скроллить таблицу построчно по вертикали, используя полосу вертикальной прокрутки и колесо.

При наличии локатора любой ввод допустимого символа или нажатие навигационных клавиш в его поле перемещает курсор и/или меняет отображения без подтверждения. Поле локатора должно редактироваться и проверяться на валидность, как поле ввода в форме «FORM» для данной таблицы при вводе/редактировании, с тем отличием, что поле локатора обрабатывается всякий раз, как меняется его содержимое. Клавиатурные комбинации для редактирования поля локатора:

Left/Right — смещение фокуса ввода на один символ;

Ctrl+Left/Ctrl+Right — смещение фокуса ввода в начало/конец поля;

BackSpace/Delete — удаление левого/правого символа;

Комбинации выделения текста;

Del — удаление выделенного текста.

Если таблица имеет локатор, фокус клавиатурного ввода всегда находится на локаторе. В противном случае фокус ввода находится на фиктивном поле, ввод в которое фильтруется и событие отправляется соответствующему элементу управления.

6) Просмотр с редактированием предполагает, что, возможно, таблица будет редактироваться. Это работает только для пользователя, имеющего право на модификацию таблицы.

Для этого используются клавиатурные команды и кнопки (button) в области функциональных действий, действующие на подсвеченную курсором строку:

Клавиатура	Кнопка или действие с указателем	Действие
Ctrl-Enter	Редактирование (Edit)	Выводит запись в форме редактирования
Insert	Вставка (Insert)	Выводит форму для вставки
Delete	Удаление (Delete)	Выводит форму для удаления
Ctrl-P	Печать (Print)	Выводит диалог печати
Enter	Двойной клик на строке	Выводит запись в форме просмотра

Если позволяет место, можно разместить кнопку «Просмотр (View)» (рисунок).

Редактирование записей в таблицах должно всегда выполняться с помощью специально для этой цели разработанных форм.

7) В режиме «просмотр без редактирования» кнопки редактирования отсутствуют, клавиатурные команды, им соответствующие, не работают, присутствует кнопка «Просмотр (View)» и «Печать (Print)».

Клавиатура	Кнопка или действие с указателем	Действие
Enter	Просмотр (View)	Выводит запись в форме просмотра
Ctrl-P	Печать (Print)	Выводит диалог печати

8) В режиме «выбор записи»⁶ работают клавиатурные команды и присутствуют кнопки:

Клавиатура	Кнопка или действие с указателем	Действие
Enter	Выбрать (Select)	Выбирает запись из таблицы

В режиме выбора записи (режим LUT) в табличной форме выводится только та информация, которая требуется для выбора. Если для принятия решения требуется дополнительная информация, она может запрашиваться отдельно и выводиться в отдельной форме.

6) в случае выбора из справочника при вводе в форму редактирования или для формирования отчета.

Особенности формы FORM

Данная форма используется для:

- 1) создания новой записи (CRF — Create Record Form);
- 2) удаления выбранной записи (DRF — Delete Record Form);
- 3) просмотра выбранной записи (VRF — View Record Form);
- 4) редактирование выбранной записи (ERF/MRF — Edit/Modify Record Form).

Основной интерфейс — клавиатура, мышка и прочие указатели работают, исключительно иницилируя клавиатурные события (это гарантирует согласованность работы клавиатуры и указателя).

Непосредственное редактирование записи в таблице, как это принято в электронных таблицах (spreadsheet), не допускается — для этого используется специальная форма редактирования, которая должна поддерживать транзакционную парадигму RMS — Read Modify Save.

Общие

Панели типа FORM в области функциональных действий из элементов управления содержат кнопки выполнения действий, а также дополнительно могут содержать необязательные строку состояния и информационную контекстную строку.

Панели типа FORM в области тела содержат следующие основные элементы управления:

- поля ввода для ввода/редактирования текста или числовых значений;
- нередатируемые поля для отображения информации, которая не может быть изменена в рамках данной формы, но требуется в качестве справки;
- выпадающие списки для выбора одного варианта из списка.
- флажки (чекбоксы) для выбора/отмены отдельных опций;
- переключатели (радиокнопки) для выбора одного варианта из группы;
- текстовые поля и области отображения информации;
- кнопки для выполнения локальных действий, например, для получения контекстной справки или расширенной информации о сущности.

Дополнительно могут присутствовать также и другие необходимые в соответствии с задачей элементы управления (контекстные меню, текстовые многострочные поля).

Все элементы управления должны иметь подписи/наименования, однозначно ассоциирующие элементы управления либо с атрибутами сущности (полями записей в БД), либо с виртуальными полями. Виртуальные поля можно визуально выделять, чтобы их отличать от полей записи.

Подписи следует располагать слева от связанных с ними элементов управления.

Элементы управления следует организовывать в именованные и/или неименованные группы.

Панели типа FORM не должны содержать элементов прокрутки, вместо этого, в случае невозможности уместить необходимую информацию в пределах одной панели, используются вкладки, организованные по группам.

Панели типа FORM по вертикали должны быть разделены на две зоны — левая содержит подписи/наименования полей, а правая — соответствующие поля.

Содержимое левой зоны выравнивается вправо, содержимое правой — влево. Таким образом создается иллюзия визуального разделителя (сплиттера) в виде пустого вертикального пространства. Также можно использовать реальные разделители, если это повышает воспринимаемость информации.

Переключатели (радиокнопки) и флажки (чекбоксы) следует объединять в группы, в этом случае слева может находиться подпись/наименование группы, а справа элементы с текстовым описанием выбора или флажка. Иногда, если позволяет место, можно размещать переключатели и флажки горизонтально.

Справа от полей ввода могут находиться кнопки для вызова дополнительной связанной с данным полем информации.

Связка «Подпись : Значение поля» должна занимать отдельную строку, по возможности, одну. Если текст «Значение поля» не уместается в одной строке, он может вводиться/выводиться в текстовом поле, обеспечивающем многострочное представление.

Справа от строки «Подпись» перед полем допускается размещение кнопки с пиктограммой, нажатие на которую вызывает модальное окно с подсказкой или дополнительной информацией (1:1).

Ниже описывается навигация по форме с использованием клавиатуры. Навигация с помощью мыши и прочих указателей вторична и должна осуществляться через вызовы и последовательности вызовов процедур обработки клавиатурных событий.

Навигация по форме

Для всех элементов управления в форме, которые могут получить клавиатурный фокус ввода, устанавливается порядок (порядок формы).

Изначально после открытия формы фокус ввода устанавливается на одном из полей в порядке формы. Обычно это одно из полей ввода.

Нажатие клавиш TAB/Shift+TAB перемещает фокус ввода вперед/назад в порядке формы, не выходя из области тела, содержимое полей ввода и их состояние при этом не меняется.

Поля, на которые можно установить фокус ввода, в общем случае могут располагаться на панели в двух измерениях (визуально выше/ниже/левее/правее).

Нажатие клавиш со стрелками перемещает фокус ввода в порядке визуального расположения элементов на панели, не выходя из области тела, содержимое полей ввода при этом не меняется.

Принятие формы — фиксация значений всех полей и выполнение соответствующих действий, изменяющих состояние базы данных.

Отказ формы — отказ от модификации всех полей и возврат в состояние, предшествующее вызову процедуры формы.

Для отказа (возврата без сохранения изменений) используется Ctrl+Esc.

Отказ формы может последовать в любой момент времени между активацией формы к вводу и ее принятием, соответственно, он не только не должен менять состояние базы данных, он не должен изменять состояние прикладной клиентской программы. Т.е. поведение формы должно быть транзакционным.

Для принятия/отказа поля ввода используются Enter/Esc, соответственно.

Для перехода из области тела панели в область вкладок обычно используется либо последовательное перемещение фокуса на одну из вкладок, либо прямая его установка.

Для прямой установки используется Ctrl+n, где n — цифра от 1 до 9 (номер вкладки).

Для последовательной установки используется Ctrl+Alt+Left/Right, который перемещает фокус выбора вкладок влево/вправо. Если вкладки располагаются по вертикали, используются Ctrl+Alt+Up/Down. При первом нажатии Ctrl+Alt+Left/Right фокус переходит на одну из вкладок, клавиша Enter или Space используется для выбора вкладки, после чего фокус ввода устанавливается на первое в порядке формы поле новой вкладки.

Для принятия (завершения, подтверждения) поля ввода используется Enter. Фокус ввода при этом перемещается в следующее поле в порядке формы.

Для отказа поля используется Esc. При нажатии Esc восстанавливается содержимое этого поля и то состояние, которое существовало на момент переноса в него фокуса. Фокус ввода при этом остается в поле.

Для печати отчета по форме обычно используется Ctrl+P.

Обработка полей ввода

Все текстовые (цифровые тоже текстовые) поля ввода обеспечивают фильтрацию по регехр по мере ввода и редактирования введенного, а также индикацию несовпадений.

В текстовом поле помимо алфавитно-цифровых клавиш и клавиш редактирования действуют две клавиши:

Enter — поле принимается;

Esc — поле отвергается.

После принятия поля по Enter его содержимое проверяется на установленные ограничения. Если ограничения удовлетворены, поле фиксируется в приложении. Поля, зависящие от принятого, переустанавливаются и фокус переходит к следующему полю в порядке формы.

Ограничения типа DEFAULT, ограничения-проверки (CHECK), NOT NULL могут и должны проверяться при завершении поля без обращения к базе данных.

Ограничения уникальности и внешнего ключа в общем случае требуют обращения к базе данных.

Обычно на полях, которые обрабатываются в рамках формы ввода/редактирования/просмотра, ограничения уникальности и внешнего ключа не используются — первичные ключи устанавливаются приложением или базой данных с соблюдением указанных ограничений только после принятия всей формы и полной проверки ограничений.

Что касается внешних ключей, есть нюансы. Таблицы-справочники (LUT) в подавляющем большинстве имеют размеры, допускающие их загрузку в память приложения. Соответственно, проверка на существование и подстановка в поле наименования не требует обращения к БД.

Поскольку создание, просмотр и редактирование формы всегда предполагает наличия в ней вместо `foreign_id` связанного с ним наименования (`name`), то предварительная загрузка справочников, на которые ссылается обрабатываемая таблица, существенно сократит нагрузку на сервер и сеть.

Если поле внешнего ключа не проходит проверку, вызывается табличная форма в режиме выбора для соответствующей таблицы-справочника. Эта форма может быть завершена с выдачей выбранной записи или с возвратом без выбора. В первом случае `name` размещается в поле формы, а `id` в структуре записи, подлежащей записи в БД.

При нажатии `Ctrl+Enter` в любом состоянии полей формы все поля в порядке формы проверяются на ограничения, которые можно проверить без обращения к базе данных. В случае успеха записи, которые затрагиваются формой, записываются в БД. В процессе могут сыграть ограничения, которые не проверялись локально.

Алгоритм записи изменений включает проверку на изменение записей после их загрузки для модификации. В ряде случаев такое поведение можно реализовать с помощью суб-транзакции типа RMS, но если это можно сделать явно в приложении, просто прочитав запись в режиме RMS в рамках существующей транзакции, то лучше поступить именно так, поскольку это создает меньшую нагрузку на БД, да и явное всегда лучше неявного и исключений не бывает.

В случае изменения записи в БД третьей стороной, пока мы ее редактировали, форма завершается с ошибкой и, если принято решение открыть ее для редактирования с учетом новых изменений, она открывается (или перезагружается) с содержимым полей, полученным в последнем чтении. Если записи не были изменены третьей стороной, они записываются в режиме транзакции.

Для переключения чекбоксов в группе используются клавиши со стрелками (навигация в пределах группы) и Space (установка). В группе чекбоксов клавиши со стрелками теряют способность к навигации в пределах формы, соответственно для выхода из группы с завершением и фиксацией используется Enter, который фиксирует значение и перемещает фокус в следующую группу или поле в порядке формы. Для выхода из группы без фиксации следует использовать TAB/Shift+TAB.

Поля ввода и флажки тоже можно размещать в группах.

Создание новой записи

Форма создания новой записи (CRF — Create Record Form) содержит все поля записи, значения в которых требуется установить, как поля ввода, так и информационные/вычисляемые.

Поле первичного ключа в форме отсутствует — ключ всегда синтетический, устанавливается приложением (или внутренними механизмами базы данных) без участия пользователя, соответственно, нет необходимости ни вводить его, ни видеть.

Использовать в качестве первичного ключа комбинацию информационных полей запрещается — только синтетические ключи, поскольку это обеспечивает независимость от изменения законодательства, изменения внутренних стандартов предприятия и базисов начальства.

Если таблица, для которой создается запись, имеет подчиненную таблицу в связи 1:1, форма CRF должна иметь возможность при необходимости создать запись для подчиненной таблицы. Вставка таких записей должна происходить как многотабличная транзакция.

Каждое поле, как поле ввода, так и информационное/вычисляемое, должно сопровождаться соответствующим текстом.

Пример CRF для таблицы employee, приведенной на рисунке 1.

Создать служащего

Фамилия, имя и отчество

Иванов Сидор Петрович

Учетный номер

ГБУ-01803542-И2-17

Должность

Первый зам. Министра образования

☒

Дата приема

12.05.2015

Класс служащего

03

Общие данные

Дата рождения

30.02.1980

Пол

муж.

Персональный номер

Номер соцстраха

Номер паспорта

MP123456789

Отказаться

Создать

Печатать карточку

Справка

Рисунок 4 — Форма ввода данных (CRF)

– никаких ночных и прочих темных схем оформления не допускается. В идеале представление всех элементов интерфейса должно настраиваться, но это необязательно. Цвет общего фона должен иметь насыщенность не более 7% (желательно полностью нейтральный) и яркость не менее 75%. Текст и фон должны быть контрастными, в том числе и при монохромном восприятии. Палитра – стандартная из 16 цветов или близкая к ней.

На кнопках желательно выполнять надписи, а не пиктограммы, чтобы пользователю было всегда понятно, какой выбор он делает. Пиктограмма может быть изменена по прихоти инклюзивного дизайнера и ввести в заблуждение, а с текстом все ясно – он вчера, сегодня и завтра будет обозначать одно и то же. Цветовое решение должно быть достаточно контрастным, чтобы в монохромном режиме все элементы панели были четко видны – это позволит работать с приложением даже дальтоникам. Цветовое решение должно предполагать, что работа с приложением выполняется только в условиях дневного освещения

Пример CRF для таблицы employee, приведенной на рисунке 1.

Просмотр данных о служащем

Фамилия, имя и отчество

Учетный номер

Должность



Дата приема / увольнения /

Класс служащего

Общие данные

Дата рождения

Пол

Персональный номер

Номер соцстраха

Номер паспорта



Изменить данные о служащем

Фамилия, имя и отчество: Иванов Сидор Петрович

Учетный номер: ГБУ-01803542-И2-17

Должность: Первый зам. Министра образования



Дата приема / увольнения: 12.05.2015 /

Дата рождения: 30.02.1980

Персональный номер: 12313123123

Номер соцстраха: 12313123123

Номер паспорта: МР123456789



Класс служащего: 03

Пол: муж.

Печатать карточку: x

Отказаться

Изменить

Справка

